

# P2389R0

## dextents Index Type Parameter

Published Proposal, 2024-02-15

**Author:**

[Bryce Adelstein Lelbach \(he/him/his\)](#) (NVIDIA)

**Source:**

[GitHub](#)

**Issue Tracking:**

[GitHub](#)

**Project:**

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

**Audience:**

WG21

---

## Abstract

[P2299R3] added `dextents` to make it less verbose to use `mdspan` in common cases. Later, an index type parameter was added to `dextents`, which increases its verbosity, defeating its original purpose. We should fix this.

## Table of Contents

### 1 Background

### 2 Problem

### 3 Proposed Changes

#### References

Informative References

## § 1. Background

[\[P0009R18\]](#) added `mdspan`, a non-owning multidimensional span abstraction to the C++ Standard Library. It is excellent and flexible, allowing users to customize data layout, access method, and index type. However, this flexibility often comes with verbosity.

The length of each dimension (the extent) of an `mdspan` are represented by an `extents` object, and each extent may be expressed either statically or dynamically:

```
// All static.
mdspan<int, 64, 64, 64> a(d);

// All dynamic.
mdspan<int, dynamic_extent, dynamic_extent, dynamic_extent> a(d, 64, 64, 64)

// Mixed static and dynamic.
mdspan<int, 64, dynamic_extent, 64> a(d, 64);
```

[\[P2299R3\]](#) sought to make `mdspans` easier to work with for one of the most common cases - when all of your extents are dynamic. It added deduction guides to make class template argument deduction (CTAD) work for `mdspan` and `extents`.

```
mdspan a(d, 64, 64); // All dynamic.
```

However, CTAD does not help in all situations. If you are declaring a member of a class or a parameter to a function, you cannot use CTAD.

```
struct X {
    std::mdspan<int, std::dynamic_extent, std::dynamic_extent, std::dynamic_ex
};

void f(std::mdspan<int, std::dynamic_extent, std::dynamic_extent, std::dynam
```

To simplify these cases, [\[P2299R3\]](#) also added `dextents<N>`, a template alias for an `extents` with `N` dynamic extents.

```
template <std::size_t N>
using dextents = /* ... */;

struct X {
    std::mdspan<int, std::dextents<3>> a;
```

```
};

void f(mdspan<int, std::dextents<3>> a);
```

## § 2. Problem

Originally, `mdspan` and `dextents` used a fixed index type (`std::size_t`). However, the signedness and size of the index type can have an impact on performance in certain cases. So, [\[P2553R1\]](#) parameterized the index type used by `mdspan` and `dextents`.

As a part of this change, an index type template parameter was added to `dextents`:

```
template <typename IndexType, std::size_t Rank>
using dextents = /* ... */
```

This change has made using `dextents` more verbose, which is unfortunate, as the main purpose of `dextents` is to make common `mdspan` usages as simple as possible:

```
struct X {
    std::mdspan<int, std::dextents<std::size_t, 3>> a;
};

void f(mdspan<int, std::dextents<std::size_t, 3>> a);
```

Index type customization is an important feature for `mdspan` to support, but it is not something that most users will need to use or think about. If they do need it, they can always use the more verbose `dextents`.

## § 3. Proposed Changes

Remove the index type parameter from `dextents` and have it default to `size_t`.

MSVC's STL and LLVM's libc++ are already shipping `dextents`. GCC's libstdc++ is not shipping `mdspan` yet.

So, modifying `dextents` would be a source-breaking change but would have no ABI impact as `dextents` is a template alias.

Alternatively and more practically, we could leave `dextents` alone, but add a new `dims` template alias that does not have an index type parameter.

## § References

### § Informative References

#### [P0009R18]

Christian Trott, D.S. Hollman, Damien Lebrun-Grandie, Mark Hoemmen, Daniel Sunderland, H. Carter Edwards, Bryce Adelstein Lelbach, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Nevin Liber. *MDSPAN*. 13 July 2022. URL: <https://wg21.link/p0009r18>

#### [P2299R3]

Bryce Adelstein Lelbach. *`mdspan`s of All Dynamic Extents*. 8 June 2021. URL: <https://wg21.link/p2299r3>

#### [P2553R1]

Christian Trott, Damien Lebrun-Grandie, Mark Hoemmen, Dan Sunderland. *Make mdspan size\_type controllable*. 16 March 2022. URL: <https://wg21.link/p2553r1>